

Stage 10: Inventory Management & Warehouse Stock Control Specification

Executive Overview

Stage 10 delivers the multi-warehouse inventory engine for the Alight International Multi-Vendor Marketplace. It guarantees concurrency-safe stock allocations, prevents overselling through checkout holds, provides automated reservation cleanup, enables multi-warehouse transfers, and produces an immutable stock movement audit ledger.

Key Features & Inventory Architecture

1. Multi-Warehouse Network

- Warehouse Profiles:** Multiple physical or 3PL fulfillment hubs per seller or platform with geolocation coordinates (latitude/longitude), contact details, primary warehouse designation, and active state toggling.
- Per-Location Stock Tracking:** Real-time stock levels partitioned across `(warehouse_id, product_id, variant_id)` tuples.
- Stock Metrics:**
 - `quantity_on_hand`: Physical stock present in warehouse.
 - `quantity_reserved`: Active checkout holds currently in progress.
 - `quantity_available`: Immediately orderable inventory ($\max(0, \text{on_hand} - \text{reserved})$).

2. Concurrency-Safe Stock Reservations

- Pessimistic Row Locking (`FOR UPDATE`):** High-traffic stock adjustments and checkout reservations lock specific stock rows to eliminate race conditions and overselling during flash sales.
- Optimistic Locking (`@Version`):** Version-stamped stock records ensure cache coherence and concurrent modification detection.
- Timed Checkout Holds:** Temporary reservation tokens (5–60 min TTL) created during cart checkout.
- Background Reclamation (`ReservationCleanupScheduler`):** Automated recurring job (every 60s) reclaims abandoned reservations back into available inventory.
- Domain Event Integration:** Emits `StockReservedEvent` on successful checkout hold for real-time inventory monitoring.

3. Inventory Movements & Audit Trail

- Transaction Types:** `INBOUND_RECEIPT`, `OUTBOUND_SALE`, `ADJUSTMENT_ADD`, `ADJUSTMENT_SUBTRACT`, `DAMAGE_WRITE_OFF`, `TRANSFER_IN`, `TRANSFER_OUT`, `RESERVATION_HOLD`, `RESERVATION_RELEASE`.

- **Immutable Ledger (`inventory_transactions`)**: Full traceability with quantity before, quantity delta, quantity after, reference ID (Order ID / Reservation Token), user actor, and audit timestamp.

4. Low Stock Alerts & Reorder Thresholds

- Automatic alert triggers when `quantity_available ≤ reorder_threshold`.
- Seller dashboard endpoint (`/api/v1/inventory/alerts/low-stock`) for replenishment tracking.

Schema Changes (`v36__inventory_management_and_stock_control.sql`)

1. **Indexes Created**: - `idx_warehouse_stock_lookup` on `warehouse_stock(warehouse_id, product_id, variant_id)` - `idx_stock_reservations_token` on `stock_reservations(reservation_token)` - `idx_stock_reservations_cleanup` on `stock_reservations(status, expires_at) WHERE status = 'PENDING'` - `idx_inventory_tx_audit` on `inventory_transactions(warehouse_id, product_id, created_at DESC)`

API Surface

Module	Method	Endpoint	Description
Warehouse	GET	<code>/api/v1/vendor/warehouses</code>	List vendor fulfillment centers
Warehouse	POST	<code>/api/v1/vendor/warehouses</code>	Register new warehouse
Warehouse	PUT	<code>/api/v1/vendor/warehouses/{id}/primary</code>	Designate primary warehouse
Inventory	GET	<code>/api/v1/inventory</code>	View all vendor inventory levels
Inventory	POST	<code>/api/v1/inventory/adjust</code>	Manual stock adjustment & PO receipt
Inventory	POST	<code>/api/v1/inventory/transfer</code>	Inter-warehouse stock transfer
Inventory	GET	<code>/api/v1/inventory/alerts/low-stock</code>	Retrieve low stock warning items
Inventory	GET	<code>/api/v1/inventory/transactions</code>	Paginated stock movement audit trail
Reservations	POST	<code>/api/v1/inventory/reservations</code>	Create temporary checkout stock hold
Reservations	POST	<code>/api/v1/inventory/reservations/{token}/confirm</code>	Finalize stock deduction upon payment
Reservations	POST	<code>/api/v1/inventory/reservations/{token}/cancel</code>	Release reservation hold

Integration Tests

- Verified in `InventoryManagementIntegrationTest.java`.